

AI Assisted Coding

Assignment –15.1

Ch. Nithin Reddy || 2303A51550 || Batch:- 8

Task 1 – Student Records API

Task:

Use AI to build a RESTful API for managing student records.

Instructions:

- Endpoints required:
 - o GET /students → List all students
 - o POST /students → Add a new student
 - o PUT /students/{id} → Update student details
 - o DELETE /students/{id} → Delete a student record
- Use an in-memory data structure (list or dictionary) to store records.
- Ensure API responses are in JSON format.

Expected Output:

- Working API with CRUD functionality for student records.

```

1 # Create a RESTful API using Python Flask for student records.
2
3 # Requirements:
4 # - In-memory storage (list/dictionary)
5 # - Fields: id, name, age, course
6 # - Endpoints:
7 # GET /students → List all
8 # POST /students → add student
9 # PUT /students/<id> → update student
10 # DELETE /students/<id> → delete student
11 # - Return JSON responses
12 # - Use proper HTTP status codes
13 # - Include error handling
14
15 # Output complete code in a single file (app.py) with run instructions.
16 from flask import Flask, request, jsonify
17 app = Flask(__name__)
18 # In-memory storage for student records
19 students = [
20     {'id': 1, 'name': 'Aarav', 'age': 20, 'course': 'Computer Science'},
21     {'id': 2, 'name': 'Diya', 'age': 21, 'course': 'Data Science'},
22     {'id': 3, 'name': 'Rahul', 'age': 19, 'course': 'Mechanical Engineering'}
23 ]
24
25 # Health/root endpoint so opening the base URL does not return 404.
26 @app.route('/', methods=['GET'])
27 def home():
28     return jsonify({
29         'message': 'Student API is running',
30         'endpoints': {
31             'list_students': 'GET /students',
32             'add_student': 'POST /students',
33             'update_student': 'PUT /students/<id>',
34             'delete_student': 'DELETE /students/<id>'
35         },
36         'current_students': students
37     }), 200
38
39 # Compatibility endpoint in case /like is used by mistake.
40 @app.route('/like', methods=['GET'])
41 def like():
42     return jsonify({
43         'message': 'Use /students endpoints for student operations.'
44     }), 200
45
46 # Endpoint to get all students
47 @app.route('/students', methods=['GET'])
48 def get_students():
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89

```

Task 2 – Library Book Management API

Task:

Develop a RESTful API to handle library books.

Instructions:

- Endpoints required:
 - o GET /books → Retrieve all books
 - o POST /books → Add a new book
 - o GET /books/{id} → Get details of a specific book
 - o PATCH /books/{id} → Update partial book details (e.g., availability)
 - o DELETE /books/{id} → Remove a book

- Implement error handling for invalid requests.

Expected Output:

- Functional API with CRUD + partial updates.

```
app.py
# Create a RESTful API using Python Flask for library book management.
# Requirements:
# - In-memory storage (list/dictionary)
# - Fields: id, title, author, availability (boolean)
# Endpoints:
# - GET /books → get all books
# - POST /books → add book
# - GET /books/<id> → get specific book
# - PATCH /books/<id> → update partial details (e.g., availability)
# - DELETE /books/<id> → delete book

# Return JSON responses
# Use proper HTTP status codes
# Add error handling for invalid IDs/requests

# Output complete code in a single file (app.py) with run instructions.
from flask import Flask, request, jsonify
app = Flask(__name__)
# In-memory storage for book records
books = [
    {'id': 1, 'title': 'The Great Gatsby', 'author': 'F. Scott Fitzgerald', 'availability': True},
    {'id': 2, 'title': '1984', 'author': 'George Orwell', 'availability': True},
    {'id': 3, 'title': 'To Kill a Mockingbird', 'author': 'Harper Lee', 'availability': True}
]

# Health/root endpoint so opening the base URL does not return 404.
@app.route('/', methods=['GET'])
def home():
    return jsonify({
        'message': 'Library API is running',
        'endpoints': {
            'list_books': 'GET /books',
            'add_book': 'POST /books',
            'get_book': 'GET /books/<id>',
            'update_book': 'PATCH /books/<id>',
            'delete_book': 'DELETE /books/<id>'
        },
        'current_books': books
    }), 200

# Compatibility endpoint in case /like is used by mistake.
@app.route('/like', methods=['GET'])
def like():
    return jsonify({
        'message': 'Use /books endpoints for book operations.'
    }), 200

# Endpoint to get all books
@app.route('/books', methods=['GET'])
def get_books():
    return jsonify(books), 200

app.py
# Endpoint to add a new book
@app.route('/books', methods=['POST'])
def add_book():
    data = request.get_json()
    if not data or 'title' not in data or 'author' not in data:
        return jsonify({'error': 'Invalid input'}), 400
    book_id = len(books) + 1
    book = {
        'id': book_id,
        'title': data['title'],
        'author': data['author'],
        'availability': True
    }
    books.append(book)
    return jsonify(book), 201

# Endpoint to get a specific book
@app.route('/books/<int:id>', methods=['GET'])
def get_book(id):
    book = next((b for b in books if b['id'] == id), None)
    if not book:
        return jsonify({'error': 'Book not found'}), 404
    return jsonify(book), 200

# Endpoint to update a book
@app.route('/books/<int:id>', methods=['PATCH'])
def update_book(id):
    data = request.get_json()
    book = next((b for b in books if b['id'] == id), None)
    if not book:
        return jsonify({'error': 'Book not found'}), 404
    if not data:
        return jsonify({'error': 'Invalid input'}), 400
    book['title'] = data.get('title', book['title'])
    book['author'] = data.get('author', book['author'])
    book['availability'] = data.get('availability', book['availability'])
    return jsonify(book), 200

# Endpoint to delete a book
@app.route('/books/<int:id>', methods=['DELETE'])
def delete_book(id):
    global books
    book = next((b for b in books if b['id'] == id), None)
    if not book:
        return jsonify({'error': 'Book not found'}), 404
    books = [b for b in books if b['id'] != id]
    return jsonify({'message': 'Book deleted'}), 200

if __name__ == '__main__':
    app.run(debug=True)
```

Task 3 – Employee Payroll API

Task:

Create an API for managing employees and their salaries.

Instructions:

- Endpoints required:
 - o GET /employees → List all employees
 - o POST /employees → Add a new employee with salary details
 - o PUT /employees/{id}/salary → Update salary of an employee
 - o DELETE /employees/{id} → Remove employee from system
- Use AI to:
 - o Suggest data model structure.

o Add comments/docstrings for all endpoints.

Expected Output:

- API supporting salary management with clear documentation.

```
app.py •
186 # Create a RESTful API using Python Flask for employee payroll management.
187
188 # Requirements:
189 # - In-memory storage (list/dictionary)
190 # - Suggest a proper data model (id, name, role, salary, etc.)
191
192 # Endpoints:
193 # - GET /employees → List all employees
194 # - POST /employees → add employee with salary
195 # - PUT /employees/<id>/salary → update salary
196 # - DELETE /employees/<id> → delete employee
197
198 # - Return JSON responses
199 # - Use proper HTTP status codes
200 # - Add error handling
201 # - Include comments/docstrings for all endpoints
202
203 # Output complete code in a single file (app.py) with run instructions.
204 from flask import Flask, request, jsonify
205 app = Flask(__name__)
206 # In-memory storage for employee records
207 employees = [
208     {'id': 1, 'name': 'John Doe', 'salary': 50000},
209     {'id': 2, 'name': 'Jane Smith', 'salary': 60000},
210     {'id': 3, 'name': 'Jim Beam', 'salary': 55000}
211 ]
212 # Health/root endpoint so opening the base URL does not return 404.
213 @app.route('/', methods=['GET'])
214 def home():
215     return jsonify({
216         'message': 'Employee API is running',
217         'endpoints': {
218             'list_employees': 'GET /employees',
219             'add_employee': 'POST /employees',
220             'update_salary': 'PUT /employees/{id}/salary',
221             'delete_employee': 'DELETE /employees/{id}'
222         },
223         'current_employees': employees
224     }), 200
225 # Compatibility endpoint in case /like is used by mistake.
226 @app.route('/like', methods=['GET'])
227 def like():
228     return jsonify({
229         'message': 'Use /employees endpoints for employee operations.'
230     }), 200
231 # Endpoint to list all employees
232 @app.route('/employees', methods=['GET'])
233 def get_employees():
234     return jsonify(employees), 200
235 # Endpoint to add a new employee
236 @app.route('/employees', methods=['POST'])
237 def add_employee():
238     data = request.get_json()
239     if not data or 'name' not in data or 'salary' not in data:
240         return jsonify({'error': 'Invalid input'}), 400
241     employee_id = len(employees) + 1
242     employee = {
243         'id': employee_id,
244         'name': data['name'],
245         'salary': data['salary']
246     }
247     employees.append(employee)
248     return jsonify(employee), 201
249 # Endpoint to update the salary of an employee
250 @app.route('/employees/<int:id>/salary', methods=['PUT'])
251 def update_salary(id):
252     data = request.get_json()
253     employee = next((e for e in employees if e['id'] == id), None)
254     if not employee:
255         return jsonify({'error': 'Employee not found'}), 404
256     if not data:
257         return jsonify({'error': 'Invalid input'}), 400
258     employee['salary'] = data['salary']
259     return jsonify(employee), 200
260 # Endpoint to delete an employee
261 @app.route('/employees/<int:id>', methods=['DELETE'])
262 def delete_employee(id):
263     global employees
264     employee = next((e for e in employees if e['id'] == id), None)
265     if not employee:
266         return jsonify({'error': 'Employee not found'}), 404
267     employees = [e for e in employees if e['id'] != id]
268     return jsonify({'message': 'Employee deleted'}), 200
269 if __name__ == '__main__':
270     app.run(debug=True)
271
272
```

Task 4 – Real-Time Application: Online Food Ordering API

Scenario:

Design a simple API for an online food ordering system.

Requirements:

- Endpoints required:

o GET /menu → List available dishes

o POST /order → Place a new order

o GET /order/{id} → Track order status

o PUT /order/{id} → Update an existing order (e.g.,

change items)

- o DELETE /order/{id} → Cancel an order

- AI should generate:

- o REST API code

- o Suggested improvements (like authentication, pagination)

Expected Output:

- Fully working API simulating a food ordering backend.

📄 Deliverables (For All Tasks)

1. AI-generated prompts for code and test case generation.
2. At least 3 assert test cases for each task.
3. AI-generated initial code and execution screenshots.
4. Analysis of whether code passes all tests.
5. Improved final version with inline comments and explanation.
6. Compiled report (Word/PDF) with prompts, test cases, assertions, code, and output.

```

app.py
app.py > home
# Create a RESTful API using Python Flask for an online food ordering system.
# Requirements:
# - In-memory storage
# - Data model: id, items (List), total_price, status
# Endpoints:
# - GET /menu → List dishes
# - POST /order → create order
# - GET /order/<id> → track order
# - PUT /order/<id> → update order items
# - DELETE /order/<id> → cancel order
# - Return JSON responses
# - Use proper HTTP status codes
# - Add error handling
# - Include comments/docstrings
# Also:
# - Suggest improvements (authentication, pagination)
# - Generate at least 3 assert test cases
# Output complete code in a single file (app.py) with run instructions.
from flask import Flask, request, jsonify
app = Flask(__name__)
# In-memory storage for menu items
menu = [
    {'id': 1, 'name': 'Pizza', 'price': 10},
    {'id': 2, 'name': 'Burger', 'price': 5},
    {'id': 3, 'name': 'Salad', 'price': 8}
]
# Health/root endpoint so opening the base URL does not return 404.
@app.route('/', methods=['GET'])
def home():
    return jsonify({
        'message': 'Food ordering API is running',
        'endpoints': {
            'list_menu': 'GET /menu',
            'create_order': 'POST /order',
            'track_order': 'GET /order/<id>',
            'update_order': 'PUT /order/<id>',
            'cancel_order': 'DELETE /order/<id>'
        },
        'current_menu': menu
    }), 200
# Compatibility endpoint in case /like is used by mistake.
@app.route('/like', methods=['GET'])
def like():
    return jsonify({
        'message': 'Use /menu and /order endpoints for food ordering operation'
    }), 200
# Endpoint to list dishes
@app.route('/menu', methods=['GET'])

```

```

app.py > ...
@app.route('/menu', methods=['GET'])
def get_menu():
    return jsonify(menu), 200
# Endpoint to create a new order
@app.route('/order', methods=['POST'])
def create_order():
    data = request.get_json()
    if not data or 'items' not in data:
        return jsonify({'error': 'Invalid input'}), 400
    order_id = len(orders) + 1
    order = {
        'id': order_id,
        'items': data['items'],
        'total_price': sum(item['price'] for item in data['items']),
        'status': 'pending'
    }
    orders.append(order)
    return jsonify(order), 201
# Endpoint to track an order
@app.route('/order/<int:id>', methods=['GET'])
def track_order(id):
    order = next((o for o in orders if o['id'] == id), None)
    if not order:
        return jsonify({'error': 'Order not found'}), 404
    return jsonify(order), 200
# Endpoint to update an order
@app.route('/order/<int:id>', methods=['PUT'])
def update_order(id):
    data = request.get_json()
    order = next((o for o in orders if o['id'] == id), None)
    if not order:
        return jsonify({'error': 'Order not found'}), 404
    if not data:
        return jsonify({'error': 'Invalid input'}), 400
    order['items'] = data['items']
    order['total_price'] = sum(item['price'] for item in data['items'])
    return jsonify(order), 200
# Endpoint to cancel an order
@app.route('/order/<int:id>', methods=['DELETE'])
def cancel_order(id):
    global orders
    order = next((o for o in orders if o['id'] == id), None)
    if not order:
        return jsonify({'error': 'Order not found'}), 404
    orders = [o for o in orders if o['id'] != id]
    return jsonify({'message': 'Order cancelled'}), 200
if __name__ == '__main__':
    app.run(debug=True)

```